Panic, Recover et Résilience

---

1. `panic` : Quand l'impossible ou l'irrécupérable survient

En Go, il existe une frontière stricte et infranchissable entre :

- Une **erreur opérationnelle attendue (`error`)** : un fichier manquant, une base de données injoignable, un JSON mal formé envoyé par un client, un disque plein. **Tout cela DOIT être géré avec `error`.**
- Un **état corrompu ou un bug de programmation (`panic`)** : une situation qui ne devrait jamais arriver si le code est écrit correctement, ou une impossibilité absolue de démarrer le programme.

Règle d'or en entreprise : **`panic` n'est PAS l'équivalent de `throw Exception` en Java/C#/Python.** On n'utilise jamais `panic` pour signaler une erreur métier normale à l'appelant.

1.1 Qu'est-ce qui déclenche un `panic` ?

Un `panic` peut être déclenché de deux manières :

1. **Automatiquement par le runtime Go** (équivalent d'un `SIGSEGV` / *Segmentation Fault* en C, mais intercepté proprement par le runtime) :
    - Déréférencement d'un pointeur `nil` (`*ptr` quand `ptr == nil`).
    - Accès hors limites dans un tableau ou une slice (`s[10]` quand `len(s) == 3`).
    - Écriture dans une `map` non initialisée (`nil map`).
    - Fermeture d'un channel déjà fermé (`close(ch)` deux fois).
    - *Type Assertion* directe ratée sans le booléen `, ok` (`val := x.(int)`).
2. **Manuellement par le développeur** via la fonction native `panic(valeur)` :
    - **Cas d'usage légitime n°1 (Fail-Fast au démarrage) :** Dans `main()` ou `init()`, si une configuration vitale manque au lancement du binaire (ex: impossible de compiler une expression régulière statique avec `regexp.MustCompile(...)` ou absence d'une clé de chiffrement obligatoire).
    - **Cas d'usage légitime n°2 (Assertion d'invariant interne) :** Dans un `switch` exhaustif interne où le `default:` est mathématiquement impossible à atteindre sauf bug du développeur.

Note sur la convention `Must...` : Dans la bibliothèque standard Go, toute fonction qui choisit de faire un `panic` au lieu de retourner une `error` commence obligatoirement par le préfixe **`Must`** (ex: `regexp.MustCompile`, `template.Must`). Elles ne doivent être utilisées qu'à l'initialisation globale du programme.

1.2 Que se passe-t-il mécaniquement lors d'un `panic` ?

Contrairement à un `abort()` ou un `exit(1)` brutal en C qui tue le processus instantanément sans rien nettoyer :

1. L'exécution normale de la fonction courante s'arrête **immédiatement**.
2. **Toutes les fonctions `defer` enregistrées dans la fonction courante sont dépilées et exécutées (en ordre LIFO) !**
3. Le `panic` remonte ensuite à la fonction appelante, exécute ses `defer`, et ainsi de suite jusqu'au sommet de la goroutine (`main`).
4. Si rien ne l'a arrêté en chemin, le programme s'arrête avec un code de sortie non nul (`exit status 2`) et affiche l'intégralité de la **Stack Trace** (trace d'appels) avec les numéros de lignes exacts.

```Go
package main

import "fmt"

func operationCritique() {
	defer fmt.Println("3. [DEFER] Nettoyage de operationCritique (exécuté malgré le crash !)")
	fmt.Println("2. Début de l'opération critique...")
	panic("corruption mémoire détectée dans le cache")
	fmt.Println("Cette ligne ne sera JAMAIS exécutée")
}

func main() {
	defer fmt.Println("4. [DEFER] Fermeture de main()")
	fmt.Println("1. Démarrage du serveur")
	operationCritique()
}
```

---

2. `recover` : Intercepter la chute et isoler les pannes

Dans un serveur de production (API HTTP, démon gRPC, Worker Pool), si une requête d'un client déclenche un bug imprévu (ex: un pointeur `nil` sur un cas limite mal testé), **il est hors**

de question que le serveur entier plante et coupe le service pour les 10 000 autres clients connectés !

Pour contenir l'explosion, Go fournit la fonction native **`recover()`**.

2.1 Les règles strictes de fonctionnement de `recover()`

`recover()` permet de stopper la remontée d'un `panic` et de reprendre le contrôle normal de l'exécution, mais elle obéit à une règle absolue :

- **`recover()` ne fonctionne QUE si elle est appelée directement à l'intérieur d'une fonction `defer` !**
- Si vous appelez `recover()` dans le flux normal d'une fonction (hors `defer`), ou s'il n'y a aucun `panic` en cours, `recover()` renvoie simplement `nil` et ne fait rien.

```
Go
func executerEnSecurite() {
	// Le bloc de sauvetage DOIT être une closure enregistrée avec 'defer' :
	defer func() {
		if r := recover(); r != nil {
			// 'r' est de type 'any' : il contient la valeur passée à
panic(...)
			fmt.Printf("[ALERTE CRASH INTERCEPTÉ] Cause : %v\n", r)
		}
	}()

	fmt.Println("Exécution d'un traitement risqué...")
	var ptr *int
	*ptr = 42 // Déclenche un runtime panic (nil pointer dereference) !
}

func main() {
	executerEnSecurite()
	fmt.Println("Le programme continue normalement après le crash isolé !")
}
```

2.2 Convertir un `panic` en valeur `error` grâce aux retours nommés

Comment faire pour qu'une fonction qui subit un `panic` interne retourne proprement une `error` standard à son appelant ? Puisqu'une fonction `defer` s'exécute **après** que le `panic` a interrompu le corps de la fonction mais **avant** que la fonction ne rende la main à l'appelant, la closure `defer` peut **modifier directement la variable de retour nommée `(err error)`** !

```
Go
package main
```

```
import (
	"fmt"
	"runtime/debug"
)

// Notez la valeur de retour nommée '(resultat int, err error)' !
func CalculProtégé(a, b int) (resultat int, err error) {
	defer func() {
		if r := recover(); r != nil {
			// On capture le crash ET la Stack Trace complète
(runtime/debug),
			// et on les assigne à la variable de retour 'err' !
			err = fmt.Errorf("crash interne intercepté (%v)\nStack:\n%s",
r, debug.Stack())
		}
	}()

	// Si b == 0, la division entière en Go déclenche un runtime panic :
	return a / b, nil
}

func main() {
	res, err := CalculProtégé(100, 0)
	if err != nil {
		fmt.Println("Erreur gérée proprement :", err)
		return
	}
	fmt.Println("Résultat :", res)
}
```

2.3 Règle de confinement : Ne laissez jamais `panic` traverser une frontière de package

Une bibliothèque bien écrite en Go ne laisse jamais un `panic` sortir de ses fonctions publiques vers le code de l'utilisateur :

1. À l'intérieur d'un même package complexe (ex: un parseur JSON récursif comme `encoding/json`), les développeurs Go utilisent parfois un `panic` privé pour sortir instantanément de 12 appels récursifs imbriqués, **mais la fonction publique d'entrée possède toujours un `defer / recover`** qui transforme ce `panic` en une `error` classique pour l'appelant.
2. **Attention aux Goroutines (Teaser Jour 3) :** Un `defer / recover` placé dans `main()` **ne peut PAS** intercepter un `panic` qui se produit dans une autre goroutine

lancée en parallèle ! Chaque goroutine possède sa propre pile d'exécution et doit embarquer son propre mécanisme de `recover`.

---

3. Agrégation d'erreurs multiples : `errors.Join` et `defer Close()`

3.1 Le problème des erreurs multiples (Go 1.20+)

Dans de nombreux cas réels, une fonction doit agréger plusieurs erreurs indépendantes :

- Validation d'un formulaire ou d'une configuration (on veut renvoyer **toutes** les erreurs de validation d'un coup, pas seulement la première).
- Fermeture d'un pool de 5 connexions réseau (si 2 connexions échouent à se fermer, on veut remonter les 2 erreurs).
- Une erreur survient pendant l'écriture dans un fichier **ET** une seconde erreur survient lors du `defer fichier.Close()`.

Depuis Go 1.20, la bibliothèque standard fournit **`errors.Join(errs ...error)`** :

```
Go
func ValiderConfig(hote string, port int, timeout int) error {
	var listeErreurs []error

	if hote == "" {
		listeErreurs = append(listeErreurs, ErrHoteVide)
	}
	if port <= 0 || port > 65535 {
		listeErreurs = append(listeErreurs, ErrPortInvalide)
	}
	if timeout < 0 {
		listeErreurs = append(listeErreurs, ErrTimeoutNegatif)
	}

	// errors.Join ignore automatiquement les éléments nil.
	// Si TOUS les éléments sont nil (ou si la slice est vide), errors.Join
retourne nil !
	return errors.Join(listeErreurs...)
}
```

La magie d'`errors.Join` avec `errors.Is` et `errors.As`

L'erreur composite retournée par `errors.Join` implémente la méthode spéciale `Unwrap() []error` (arbre d'erreurs). Par conséquent, **`errors.Is` et `errors.As` parcourent automatiquement toutes les branches de l'erreur jointe !** Si `err :=`

`ValiderConfig("", -80, 10)` échoue sur l'hôte et le port, alors `errors.Is(err, ErrHoteVide)` **ET** `errors.Is(err, ErrPortInvalide)` renverront tous les deux `true` !

3.2 Le piège professionnel du `defer fichier.Close()` en écriture

En lecture seule, écrire `defer f.Close()` en ignorant la valeur de retour de `Close()` est toléré. Mais en **écriture** (sur un fichier ou un flux réseau bufferisé), la fonction `Write()` se contente souvent de copier les octets dans un tampon en RAM. C'est l'appel final à `Close()` qui vide réellement le tampon sur le disque (*flush*). Si le disque est plein au moment du `Close()`, et que vous avez écrit `defer f.Close()`, **l'erreur est silencieusement perdue et le fichier est corrompu !**

Voici le pattern de production combinant retour nommé, `defer` et `errors.Join` :

```Go
func SauvegarderDonnees(chemin string, donnees []byte) (err Finale error) {
	f, err := os.Create(chemin)
	if err != nil {
		return err
	}

	// Ce defer capture l'erreur éventuelle de Close() et la fusionne avec errFinale !
	defer func() {
		errClose := f.Close()
		errFinale = errors.Join(errFinale, errClose)
	}()

	_, err = f.Write(donnees)
	return err // Sera assigné à 'errFinale' AVANT l'exécution du defer !
}
```